15-Day Python to Web Development & Data Visualization Practice Plan

Goal

By the end of 15 days, students should have hands-on practice with:

- Basic to advanced Python
- Regular Expressions (Regex)
- Object-Oriented Programming (OOP)
- FastAPI basics
- NumPy
- Pandas
- Matplotlib
- Seaborn
- Plotly
- Django basics
- GitHub/project documentation
- Professional LinkedIn posting

Daily Practice Rule

Every day, students should:

- [ ] Learn the assigned concepts.
- [ ] Write code without simply copying tutorials.
- [ ] Complete the day's exercises.
- [ ] Save the work in a GitHub repository.
- [ ] Write a short summary of what they learned.
- [ ] Post their progress on LinkedIn.
- [ ] Mention practical challenges and how they solved them.

Day 1 — Python Fundamentals

Topics

- Python syntax
- Variables and data types

- Input/output
- Operators
- Type conversion
- Strings
- Basic string methods
- Comments and code formatting

Practice

- Create a calculator.
- Check whether a number is even or odd.
- Convert Celsius to Fahrenheit.
- Count characters in a string.
- Reverse a string.
- Create a simple student-information program.

Deliverable

Create `day01_python_basics.py` containing at least **10 small programs**.

Day 2 — Conditions, Loops & Functions

Topics

- `if`, `elif`, `else`
- `for` loops
- `while` loops
- `break`, `continue`, `pass`
- Functions
- Parameters and arguments
- Return values
- Scope

Practice

Build:

- Number guessing game
- Multiplication table generator
- Prime-number checker
- Factorial program
- Fibonacci sequence
- Password validation program

Deliverable

Create at least **8 practice programs** using conditions, loops and functions.

Day 3 — Python Data Structures

Topics

- Lists
- Tuples
- Sets
- Dictionaries
- List/dictionary methods
- Nested data structures
- List comprehensions
- Dictionary comprehensions

Practice Project

Create a **Student Management Program**.

It should allow users to:

- Add students
- Store marks
- Calculate average marks
- Find the highest scorer
- Search for a student
- Display student information

Deliverable

A command-line student management program.

Day 4 — Intermediate & Advanced Python

Topics

- `*args` and `**kwargs`
- Lambda functions
- `map()`

- `filter()`
- `reduce()`
- Iterators
- Generators
- `yield`
- Decorators
- Exception handling

Practice

Implement:

- Custom decorator for logging
- Generator for Fibonacci numbers
- Generator for even numbers
- Exception-safe calculator
- Function using `*args` and `**kwargs`

Challenge

Build a **custom expense tracker** using functions, exception handling and data structures.

Day 5 — File Handling, Modules & Regex

Topics

- Reading files
- Writing files
- CSV/JSON basics
- Modules and packages
- `import`
- Regular expressions
- `re.search()`
- `re.match()`
- `re.findall()`
- `re.sub()`

Regex Practice

Create patterns for:

- Email validation
- Phone number validation
- Password validation

- URL extraction
- Finding hashtags
- Extracting numbers from text

Mini Project

Build a **Text/Data Extractor** that reads a text file and extracts emails, phone numbers and URLs.

Day 6 — Object-Oriented Programming

Topics

- Classes and objects
- Constructors
- Instance attributes
- Class attributes
- Instance methods
- Class methods
- Static methods
- Encapsulation

Practice Project

Create a **Bank Management System**.

Include:

- Customer class
- Account class
- Deposit
- Withdrawal
- Balance checking
- Transaction history

Deliverable

Demonstrate proper use of classes, objects and methods.

Day 7 — Advanced OOP

Topics

- Inheritance
- Multiple inheritance
- Method overriding
- Polymorphism
- Abstraction
- Abstract classes
- Magic/dunder methods
- Composition

Practice Project

Build a **Vehicle Management System**.

Create classes such as:

- Vehicle
- Car
- Bike
- Truck

Implement inheritance and polymorphism.

Challenge

Explain in your README where you used:

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

Day 8 — NumPy

Topics

- NumPy arrays
- Array creation
- Dimensions
- Shape and size
- Indexing and slicing
- Reshaping

- Mathematical operations
- Broadcasting
- Aggregation functions
- Random numbers

Practice

Work with a dataset represented as NumPy arrays.

Calculate:

- Mean
- Median
- Standard deviation
- Minimum
- Maximum
- Sum

Mini Project

Create a **Student Marks Analysis** using NumPy.

Day 9 — Pandas

Topics

- Series
- DataFrame
- Reading CSV files
- Selecting rows/columns
- Filtering
- Sorting
- Missing values
- `groupby()`
- Aggregation
- Merging datasets

Practice Project

Use a real-world CSV dataset.

Perform:

- Data loading
- Data inspection
- Missing-value handling
- Filtering
- Sorting
- Grouping
- Statistical analysis

Deliverable

Create a Jupyter Notebook with your complete analysis.

Day 10 — Matplotlib & Seaborn

Matplotlib Topics

- Line charts
- Bar charts
- Histograms
- Scatter plots
- Pie charts
- Labels
- Titles
- Legends
- Subplots

Seaborn Topics

- Statistical visualization
- Count plots
- Box plots
- Violin plots
- Heatmaps
- Pair plots
- Distribution plots

Practice Project

Create a **Data Visualization Report** containing at least:

- 2 Matplotlib charts
- 4 Seaborn charts
- A short explanation of every visualization

Day 11 — Plotly & Interactive Visualization

Topics

- Plotly basics
- Interactive charts
- Bar charts
- Line charts
- Scatter plots
- Pie charts
- Interactive hover information
- Basic dashboard concepts

Practice Project

Create an **Interactive Data Visualization Dashboard**.

Include:

- Interactive bar chart
- Interactive line chart
- Scatter plot
- Filters where applicable
- Meaningful titles and labels

Deliverable

Publish/document the dashboard and explain what insights you found.

Day 12 — FastAPI Basics

Topics

- What is an API?
- REST API concepts
- FastAPI installation
- Routes
- GET requests
- POST requests
- Path parameters
- Query parameters

- Request bodies
- Pydantic models
- Automatic API documentation

Practice Project

Build a **Student API**.

Endpoints:

- `GET /students`
- `GET /students/{id}`
- `POST /students`
- `PUT /students/{id}`
- `DELETE /students/{id}`

Deliverable

A working FastAPI application with documented endpoints.

Day 13 — Django Basics

Topics

- What is Django?
- Django project structure
- Applications
- URLs
- Views
- Templates
- Models
- Migrations
- Django Admin
- Basic CRUD

Practice Project

Create a **Student Management Website**.

Features:

- Student list
- Student detail page
- Add student

- Edit student
- Delete student
- Django Admin integration

Day 14 — Full Integration Project

Project: Student Analytics System

Combine the skills learned during the previous days.

Suggested Features

Python

- Data processing functions

Pandas/NumPy

- Analyze student performance

Matplotlib/Seaborn/Plotly

- Generate performance visualizations

FastAPI

- Provide student/analytics API endpoints

Django

- Create a basic web interface

Minimum Requirements

- [ ] Student data
- [ ] CRUD functionality
- [ ] Data analysis
- [ ] At least 3 visualizations
- [ ] At least 3 API endpoints
- [ ] Basic Django interface
- [ ] Clean project structure

- [ ] README documentation

Day 15 — Final Project, GitHub & LinkedIn

Final Tasks

Students should:

- [ ] Create a LinkedIn post about the 15-day journey.
- [ ] Complete their project.
- [ ] Clean and organize the code.
- [ ] Add comments where necessary.
- [ ] Create a professional `README.md`.
- [ ] Add screenshots of the project.
- [ ] Upload the project to GitHub.
- [ ] Explain the technologies used.
- [ ] Mention challenges faced.
- [ ] Mention what they learned.

Final README Structure

1. Project Title
2. Project Description
3. Features
4. Technologies Used
5. Project Structure
6. Installation Instructions
7. How to Run
8. Screenshots
9. API Endpoints
10. Future Improvements
11. What I Learned

Expected Outcome

After completing the 15 days, students should be able to:

- Write Python programs confidently.
- Use intermediate and advanced Python concepts.
- Work with Regex.
- Apply OOP concepts.
- Analyze data using NumPy and Pandas.
- Create visualizations using Matplotlib, Seaborn and Plotly.
- Build basic REST APIs using FastAPI.
- Create basic web applications using Django.
- Combine multiple technologies into a practical project.
- Maintain their work on GitHub.
- Communicate their learning professionally on LinkedIn.